

untangle::executor

Generated by Doxygen 1.18.0

1 executor	1
2 Namespace Index	7
2.1 Namespace List	7
3 Class Index	9
3.1 Class List	9
4 File Index	11
4.1 File List	11
5 Namespace Documentation	13
5.1 untangle Namespace Reference	13
5.1.1 Detailed Description	13
6 Class Documentation	15
6.1 untangle::executor< actionT > Class Template Reference	15
6.1.1 Detailed Description	15
6.1.2 Constructor & Destructor Documentation	16
6.1.2.1 executor()	16
6.1.2.2 ~executor()	16
6.1.3 Member Function Documentation	17
6.1.3.1 add_action()	17
6.1.3.2 add_task()	18
6.1.3.3 start()	19
6.1.3.4 stop()	19
6.1.4 Member Data Documentation	19
6.1.4.1 on_task_error	19
7 File Documentation	21
7.1 executor.hpp	21
Index	25

Chapter 1

executor

c++ thread pool executor based on async execution

The interface is one header that exposes a pool of workers, each one an **async** execution in continuous mode, fed by a shared queue of tasks.

This is the prototype, imported. It was written in `async/prototypes/` to answer whether execution is a usable building block for a pool — it is — and it is being hardened here. The behaviour below is what the tests state today, not a settled interface.

requirements

C++23, and the `async` submodule:

```
git submodule update --init --recursive
```

`async` carries its own actuator submodule, which is why the clone has to be recursive. Both are header-only; nothing is built or installed.

The headers report warnings with `std::println`, so they need a standard library that provides `<print>` — GCC 14 or newer.

example

```
#include <executor.hpp>
#include <functional>

int main() {
    untangle::executor<std::function<void(void)>> pool(4);
    pool.start();

    for (int i = 0; i < 100; ++i) {
        pool.add_action([i] { work(i); });
    }

    return 0; // ~executor finishes what was added, then stops the workers
}
```

the action type

The pool is a template on the callable it runs, the way an execution is — and on the same parameter name, because both kinds of queued work are that one callable. One pool runs one signature, and the caller names it:

```
untangle::executor<std::function<int(void)>> pool(4); // returns are run and dropped
```

The action type does not have to be a `std::function`. What an execution requires of it is a nested `result_type`, which it reads rather than deduces, so any callable declaring one will do — and a pool built on such a callable never touches `std::function::result_type`, which the standard removed in C++20 and both `libc++` and `libstdc++` still supply by grace rather than contract.

```
struct my_task {
    using result_type = void;
    void operator()() const;
};
```

```
untangle::executor<my_task> pool(4);
```

An action type may take arguments, and `add_action()` binds them the way its namesake `execution::add_action()` does — at the door, where the caller still holds them:

```
untangle::executor<std::function<int(int)>> pool(4);
pool.start();
pool.add_action([](int n) { return n * 2; }, 21);
```

What waits for a worker is one callable carrying its own arguments, so nothing about a task changes between the queue and the worker that runs it. The arguments are copied at `add_action()` and handed to the task as the pool's own lvalues when it runs: a task is called later, possibly much later, and what the caller passed may be gone by then. A task taking `int&` therefore mutates the pool's copy, which the caller never sees.

A task that cannot be called with the arguments given is refused by a `static_assert` in `add_action()`, rather than by an error from inside the binding.

tasks: work that reports back

Everything above queues work through `add_action()`, which is fire and forget: it runs, what it returns is dropped, and the caller hears nothing more about it. `add_task()` carries the callback it must notify:

```
untangle::executor<std::function<int(int)>> pool(4);
pool.start();

// the callback comes last, after the arguments the task is bound to
pool.add_task([](int n) { return n * 2; }, 21, [](int result) { /* result == 42 */ });
```

Work that returns void still reports that it finished, with a callback taking nothing — finished is the message and the result is optional:

```
untangle::executor<std::function<void(void)>> pool(4);
pool.add_task([] { /* ... */ }, [] { /* finished */ });
```

The callback is the last argument, and the signature cannot say so. A parameter pack cannot be followed by a deducible parameter, so it arrives inside the arguments and `untangle::bind_task()` splits it off. It must be callable with the task's result and return nothing — or callable with nothing at all, when the task returns void. A missing or unusable one is a compile error rather than silence.

Both doors share one queue, and the order out is the order in. An action posted before a task runs before it. The pool can promise that because it owns one container; `async::execution`, which the workers are, fires every action in a pass before any task, so its ordering across the two kinds differs. Only the pool's own order is a promise to a caller of the pool.

`pending()` counts both kinds, as it always counted work waiting for a worker.

A refused task does not notify. `add_task()` answers false under exactly the conditions `add_action()` does — before `start()`, after `stop()`, once the destructor has begun, or if the worker it was offered to has stopped — and a refused task is destroyed rather than queued. The answer is the whole report, so a caller waiting on the callback rather than on the answer would wait for ever. The same is true of work still queued when a stopped pool is destroyed: it never runs, so it never notifies, and the count on `stderr` is all there is.

Finished does not mean failed. A task that throws is not notified: there is no result to hand over, and for a void task no completion to report either. What it threw goes to `on_task_error`, as a failing action's does. And because a callback runs inside the same try as the task that owns it, `on_task_error` can fire for a task that in fact succeeded — the work was done and only the telling failed.

A callback runs on a worker's thread, inside that worker's drain, and several may be running at once on different workers. It is the same rule `on_task_error` carries, for the same reason: nothing may escape it.

how work is placed

N executions run in continuous mode behind a shared `std::deque` of tasks.

Nothing overtakes. A task goes straight to a free worker only when the shared queue is empty; otherwise it joins the back of the queue. Finding a free worker does not let a task jump a queue that already has work in it.

Workers pull rather than being pushed to. When one reports its queue finished — `on_finished`, raised by execution the moment its list empties — it takes the front of the shared queue. That is what spreads the work, with no scheduling logic of its own: 400 tasks over 4 workers come out 100/100/100/100.

A worker is asked, not tracked. `execution::is_busy()` is what `add_action()` reads to find a free worker and what the destructor reads to decide the pool is idle. A tally kept here would be the pool's belief about its workers; this is the workers' own answer, and it cannot drift.

running, stopping, restarting

A pool is built stopped. The constructor makes the workers and wires them up; `start()` runs them. Until it is called the pool refuses work, because a task taken then would sit unrun and the destructor would wait for it. It is also the moment to assign `on_task_error`, before anything can throw.

`stop()` stops the workers, and stops the pool accepting. What a worker is already running it finishes; what is still queued stays queued. `stop()` does not wait for the workers to leave their threads — only the destructor does that, and that wait is what makes destroying the pool safe.

`start()` after `stop()` restarts it, workers and all, and the queue it was holding drains as they pick it up again. Either call is harmless twice over.

lifetime and failure

The destructor finishes what was added. It stops accepting, waits for the queue to empty and for every worker to go idle, and only then stops them. A task added before the pool goes out of scope has run by the time it does.

A pool destroyed while stopped is the exception: its queue can never empty, because only a running worker takes from it, so the destructor does not wait for it. Whatever is left is reported on stderr and dropped.

Both waits are unbounded, and both say so while they last. A task that never returns would otherwise leave the destructor silent for ever, so it reports instead — the first wait naming who is still inside a task, the second naming whose thread has not left:

```
executor: still waiting to finish after 1s - 1 queued, 2 in a task: pool_worker_0, pool_worker_1
executor: still waiting to finish after 3s - 0 queued, 1 in a task: pool_worker_1
executor: still waiting to stop after 1s - 1 not left its thread: pool_worker_1
```

The interval starts at a second and doubles to a ceiling of thirty, so a stuck teardown says something almost at once without a long one becoming a flood. The two waits are kept apart because they answer different questions: a worker can be idle and still be in its thread.

Bounding either wait was never available. Returning early while a worker is still in a task is a use-after-free, and a bounded wait only moves the hang into `~execution()`, which spins on its own running state with no timeout. Workers are detached and `async::execution` has no cancellation, so a pool cannot outlive a task it cannot interrupt. What can be fixed is the silence.

`add_action()` says whether the task was taken. It returns false before `start()`, after `stop()`, and once the destructor has begun. It returns false again if the worker it was offered to has stopped. A refused task is destroyed rather than queued, so a false answer means it will not run.

A task that is still running is refused too, and that is chosen rather than incidental. A pool shutting down is waiting for exactly that task, so it is tempting to let the work it spawns through — but then a task that re-adds itself could keep the shutdown from ever ending. One rule instead: a pool that is not running takes no work, whoever is asking. `add_action()` answers false inside the task, so it can tell.

A task that throws does not take the worker with it, and what it threw is not lost. The worker catches it and carries on with the next task; `on_task_error` is where the throw goes:

```
untangle::executor<std::function<void(void)>> pool(4);

pool.on_task_error = [](std::exception_ptr thrown) {
    try {
        std::rethrow_exception(thrown);
    } catch (const std::exception& e) {
        log(e.what());
    }
};
```

It carries a `std::exception_ptr` because a task may throw something that is not a `std::exception`, and that is the case most worth hearing about. Assign it before the first `add_action()` — a worker reads it — and expect it on a worker's thread, with more than one worker possibly inside it at once. Leave it unset and the pool warns on stderr instead, naming the worker.

That name is not unique between pools. Workers are named by index — `pool_worker_0`, `pool_worker_1` — so two pools in one process each have a `pool_worker_0`, and every warning that names one is ambiguous: the failure reports above, and the destructor's waits alike. With a single pool the name says which worker; with two it does not. Assigning `on_task_error` sidesteps it for failures, since the handler is the pool's own and the warning is replaced.

A return value is still not collected: a continuous worker keeps none. Work with something to report carries its own channel, which is what `add_task()` is for — see tasks: work that reports back above.

building the tests

```
cmake -S test -B test/build
cmake --build test/build
ctest --test-dir test/build
```

test/CMakeLists.txt fetches googletest at configure time, so the first configure needs network access.

sanitizers

EXECUTOR_SANITIZE builds the tests under a sanitizer. It is off by default, because a sanitized build is several times slower and ThreadSanitizer does not ship for every toolchain.

```
cmake -S test -B test/build-asan -DEXECUTOR_SANITIZE=address
cmake --build test/build-asan
ctest --test-dir test/build-asan
```

Accepted values are address, thread, undefined, or empty. Anything else is refused at configure time rather than passed through to the compiler.

Use a separate build directory per sanitizer. address and thread instrument the same accesses in incompatible ways and cannot be combined, so one build directory is one sanitizer.

the reference

tools/make_doc.sh builds doc/refman.pdf from the header and this README. It needs doxygen, tectonic and python3, and it fails on any doxygen warning that is not an obsolete-tag notice, so an undocumented member or a broken \ref does not reach the reference.

Chapter 2

Namespace Index

2.1 Namespace List

Here is a list of all documented namespaces with brief descriptions:

untangle	A thread pool executor built out of <code>untangle::async::execution</code> objects	13
--------------------------	---	--------------------

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

[untangle::executor< actionT >](#)
Runs actions and tasks on a fixed pool of untangle::async::execution workers [15](#)

Chapter 4

File Index

4.1 File List

Here is a list of all documented files with brief descriptions:

executor.hpp	21
--	----

Chapter 5

Namespace Documentation

5.1 untangle Namespace Reference

A thread pool executor built out of `untangle::async::execution` objects.

Classes

- class `executor`
Runs actions and tasks on a fixed pool of `untangle::async::execution` workers.

5.1.1 Detailed Description

A thread pool executor built out of `untangle::async::execution` objects.

N workers run in continuous mode behind a shared queue. A task goes straight to a free worker only when the queue is empty; otherwise it joins the back, and a worker takes the front as it frees up.

Chapter 6

Class Documentation

6.1 `untangle::executor< actionT >` Class Template Reference

Runs actions and tasks on a fixed pool of `untangle::async::execution` workers.

```
#include <executor.hpp>
```

Public Member Functions

- `executor` (`std::size_t worker_count`)
Builds `worker_count` workers. They do not run until `start()`.
- `~executor` ()
Finishes what has been added, then stops every worker.
- `template<typename... Args>`
`bool add_action` (`actionT task`, `Args &&... args`)
Queues a task bound to `args`, or hands it to a worker that has nothing to do.
- `template<typename... Args>`
`bool add_task` (`actionT task`, `Args &&... args`)
Queues work that must report, or hands it to a worker that has nothing to do.
- `std::size_t pending` () `const`
How many tasks are waiting for a worker.
- `void start` ()
Starts every worker, and starts the pool accepting work.
- `void stop` ()
Stops every worker. What they are already running, they finish.

Public Attributes

- `std::function< void(std::exception_ptr)>` `on_task_error`
Called with whatever the work threw. Assigned by the caller.

6.1.1 Detailed Description

```
template<typename actionT>  
class untangle::executor< actionT >
```

Runs actions and tasks on a fixed pool of `untangle::async::execution` workers.

Template Parameters

actionT	The callable the pool runs. One pool runs one signature - its arguments as much as its result.
---------	--

Remarks

****Two kinds of work may be queued, and both are an actionT.**** [add_action\(\)](#) is fire and forget: the work runs, what it returns is dropped, and nothing reports that it finished. [add_task\(\)](#) carries the callback it must notify, and reports what the work produced - or, for a callable returning void, merely that it finished. The names and the split are `async::execution`'s, which this is built out of.

6.1.2 Constructor & Destructor Documentation

6.1.2.1 executor()

```
template<typename actionT>
untangle::executor< actionT >::executor (
    std::size_t worker_count) [inline], [explicit]
```

Builds `worker_count` workers. They do not run until [start\(\)](#).

Parameters

<code>worker_count</code>	- How many workers to run. Must not be zero.
---------------------------	--

Exceptions

<code>std::invalid_argument</code>	- <code>worker_count</code> is zero. Such a pool would take work it could never run.
------------------------------------	--

6.1.2.2 ~executor()

```
template<typename actionT>
untangle::executor< actionT >::~executor () [inline]
```

Finishes what has been added, then stops every worker.

Both waits are unbounded, and both report on `stderr` while they last, naming the workers they are waiting on.

Attention

A task still running here is refused if it adds more work, even though this is waiting for that very task. It is deliberate: a shutdown that took new work could be kept from ever ending by a task that re-adds itself. [add_action\(\)](#) and [add_task\(\)](#) answer false, so the task can see it.

Queued work dropped here never runs, so a task dropped here never notifies. The count on `stderr` is the whole report, and a caller waiting on a callback rather than on the answer would wait for ever. Destroying a pool with work still queued is how that happens; the destructor says how much was lost but not which.

6.1.3 Member Function Documentation

6.1.3.1 add_action()

```
template<typename actionT>
template<typename... Args>
bool untangle::executor< actionT >::add_action (
    actionT task,
    Args &&... args) [inline]
```

Queues a task bound to args, or hands it to a worker that has nothing to do.

Remarks

Named for `async::execution::add_action()`, which it forwards to and which behaves the same way: the work runs, what it returns is dropped, and nothing reports that it finished. The answer below is the last the caller hears.

The arguments are bound here, while the caller still holds them, and what waits for a worker is one callable carrying its own. That is the shape `async::execution::add_action()` offers at its door, and it is bound here for the same reason it is bound there: a queue holds callables, and a call site cannot be queued.

Remarks

The first free worker takes it, counting from the start, so a pool that is not busy keeps giving work to the same one. That is the intent: the worker that just ran a task is the warm one, and a free worker is free whichever it is.

Template Parameters

Args	- The argument types to bind to task.
------	---------------------------------------

Parameters

task	- The task to run.
args	- What to bind to it. Copied here and handed to the task as the pool's own lvalues when it runs, so a task taking a reference is given that copy rather than the caller's object, which may be gone by then.

Returns

true - the task was accepted. false - it was refused, by a pool that has not been started, has been stopped, or is being destroyed, or by a worker that has stopped. Either way it will not run. A task calling this from a worker thread is answered like any other caller.

6.1.3.2 add_task()

```
template<typename actionT>
template<typename... Args>
bool untangle::executor< actionT >::add_task (
    actionT task,
    Args &&... args) [inline]
```

Queues work that must report, or hands it to a worker that has nothing to do.

The task runs exactly as one given to [add_action\(\)](#) does, and then the callback it was built with is invoked with what it returned - on the worker's thread, inside the worker's drain.

Attention

The last argument is the callback, and this signature cannot say so: a parameter pack cannot be followed by a deducible parameter, so it arrives inside args and untangle::bind_task() splits it off. It must be callable with the task's result and return nothing - or callable with nothing at all, when the task returns void, because finished is the message and the result is optional. A missing or unusable one is a static_assert rather than silence.

Finished does not mean failed. A task that throws is not notified - there is no result to hand over - and what it threw goes to [on_task_error](#), as a failing action's does. A caller waiting on the callback alone would wait for ever.

A refused task does not notify either. The answer below is the whole report.

Remarks

It shares one queue with [add_action\(\)](#), and the order is the order they arrived: an action posted before a task runs before it.

Template Parameters

Args	- The argument types to bind, of which the last is the callback.
------	--

Parameters

task	- The task to run.
args	- What to bind to it, followed by the callback to notify. Copied here, as add_action() copies them, and for the same reason.

Returns

true - the task was accepted, and the callback will be notified when it has run.

false - refused, exactly as [add_action\(\)](#) is refused, and it will neither run nor notify.

6.1.3.3 start()

```
template<typename actionT>
void untangle::executor< actionT >::start () [inline]
```

Starts every worker, and starts the pool accepting work.

A pool refuses work until this is called, and refuses it again after [stop\(\)](#) until this is called a second time - a stopped pool is restarted by it, workers and all. Calling it twice over is harmless.

6.1.3.4 stop()

```
template<typename actionT>
void untangle::executor< actionT >::stop () [inline]
```

Stops every worker. What they are already running, they finish.

Queued tasks stay unrun and a task added afterwards is refused. That holds whoever is asking, a task already running on a worker included - a pool that is not running takes no work. [start\(\)](#) gives the workers a new working life. Calling it twice is harmless.

Attention

It does not wait for the workers to leave their threads. Only the destructor does that, and that wait is what makes destroying the pool safe.

6.1.4 Member Data Documentation

6.1.4.1 on_task_error

```
template<typename actionT>
std::function<void(std::exception_ptr)> untangle::executor< actionT >::on_task_error
```

Called with whatever the work threw. Assigned by the caller.

Nothing else reports failed work: [add_action\(\)](#) and [add_task\(\)](#) have both answered long before it runs. Left unset, the pool warns on stderr instead.

Remarks

It receives a `std::exception_ptr`, because the work may throw what is not a `std::exception`. Rethrow it to read it.

A task's failure arrives here as well, and so does its callback's. A task that throws is not notified - there is no result to hand over - so this is the only report it makes. And because a callback runs inside the same try as the task that owns it, this can fire for a task that in fact succeeded: the work was done and only the telling failed.

Attention

It runs on a worker's thread, and several workers may be in it at once. Assign it before the first [add_action\(\)](#) or [add_task\(\)](#), and let nothing escape it.

A task's callback runs on a worker's thread too, under the same rule and for the same reason - several tasks may be notifying at once, on different workers, and nothing may escape a callback either.

The documentation for this class was generated from the following file:

- `executor.hpp`

Chapter 7

File Documentation

7.1 executor.hpp

```
00001 // Copyright (c) 2026 Nicolae Popescu. MIT License.
00002
00010 #pragma once
00011
00012 #include <algorithm>
00013 #include <async.hpp>
00014 #include <atomic>
00015 #include <chrono>
00016 #include <condition_variable>
00017 #include <cstdint>
00018 #include <deque>
00019 #include <exception>
00020 #include <functional>
00021 #include <memory>
00022 #include <mutex>
00023 #include <print>
00024 #include <stdexcept>
00025 #include <string>
00026 #include <thread>
00027 #include <type_traits>
00028 #include <utility>
00029 #include <vector>
00030
00031 namespace untangle {
00032
00045 template <typename actionT>
00046 class executor {
00047 public:
00056 explicit executor(std::size_t worker_count) {
00057     if (worker_count == 0) {
00058         throw std::invalid_argument("executor: worker_count must not be zero");
00059     }
00060
00061     workers_.reserve(worker_count);
00062
00063     for (std::size_t index = 0; index < worker_count; ++index) {
00064         auto next = worker_execution::create_instance("pool_worker_" + std::to_string(index));
00065
00066         // Tells the pool the moment this worker has room for more.
00067         next->on_finished = [this, index] { take_next_task(index); };
00068
00069         // Carries what a task throws to the caller. Assigned here because a worker thread reads it.
00070         next->on_error = [this, name = next->name](std::exception_ptr thrown) {
00071             report_task_error(name, thrown);
00072         };
00073
00074         // Adds the worker to the pool the destructor waits on.
00075         poll_.add(*next);
00076
00077         workers_.push_back(std::move(next));
00078     }
00079 }
00080
00097 ~executor() {
00098     // Whether the pool was still running when it was destroyed. A stopped one cannot empty its
00099     // queue - only a worker takes from it, through on_finished, and a stopped worker raises no more
00100     // - so its queue is not waited for.
```

```

00101     const bool was_running = running_.exchange(false);
00102
00103     {
00104         std::unique_lock<std::mutex> lock(mutex_);
00105
00106         // Waits for the queue to empty and every worker to go idle.
00107         auto interval = std::chrono::milliseconds(report_first_ms);
00108         auto waited = std::chrono::milliseconds(0);
00109
00110         while (!finished_cv_.wait_for(lock, interval, [this, was_running] {
00111             return nothing_running() && (pending_.empty() || !was_running);
00112         })) {
00113             waited += interval;
00114
00115             // Under the lock, so the counts agree with the predicate that just failed.
00116             std::println(
00117                 stderr, "executor: still waiting to finish after {}s - {} queued, {} in a task{}",
00118                 waited.count() / 1000, pending_.size(), count_workers(&worker_execution::is_busy),
00119                 name_workers(&worker_execution::is_busy));
00120
00121             interval = std::min(interval * 2, std::chrono::milliseconds(report_max_ms));
00122         }
00123
00124         if (!pending_.empty()) {
00125             std::println(stderr,
00126                 "executor: {} queued task(s) dropped, the pool was stopped before they ran",
00127                 pending_.size());
00128         }
00129     }
00130
00131     stop();
00132
00133     // Waits for every worker to leave its thread, which is what makes destroying them safe. A
00134     // different question from the wait above: a worker can be idle and still be in its thread.
00135     auto interval = std::chrono::milliseconds(report_first_ms);
00136     auto waited = std::chrono::milliseconds(0);
00137
00138     while (poll_.is_running()) { // time of check
00139         const auto tick = std::chrono::milliseconds(poll_interval_ms);
00140         std::this_thread::sleep_for(tick);
00141         waited += tick;
00142
00143         if (waited >= interval) {
00144             std::println(stderr, "executor: still waiting to stop after {}s - {} not left its thread{}",
00145                 waited.count() / 1000, count_workers(&worker_execution::is_running),
00146                 name_workers(&worker_execution::is_running));
00147
00148             interval = std::min(interval * 2, std::chrono::milliseconds(report_max_ms));
00149         }
00150     }
00151
00152     // time of use: no worker thread is left to reach workers_.
00153     workers_.clear();
00154 }
00155
00182 template <typename... Args>
00183 bool add_action(actionT task, Args&&... args) {
00184     // The call the pool will make, which is not quite the one written at the call site: what
00185     // reaches the task is the copies bound below, so that is the call that has to compile.
00186     static_assert(std::is_invocable_v<actionT, std::decay_t<Args>&...>,
00187         "executor: the task cannot be called with the arguments given to add_action()");
00188
00189     if (!running_) {
00190         return false;
00191     }
00192
00193     // Bound before the lock: it copies the caller's arguments, and nothing here needs the queue.
00194     return queue(bind_action(std::move(task), std::forward<Args>(args)...));
00195 }
00196
00229 template <typename... Args>
00230 bool add_task(actionT task, Args&&... args) {
00231     if (!running_) {
00232         return false;
00233     }
00234
00235     // untangle::bind_task() takes the callback off the end of the pack and checks it; what comes
00236     // back is the bound call and the callback, which the pool carries side by side in its queue.
00237     auto built = untangle::bind_task(std::move(task), std::forward<Args>(args)...);
00238
00239     return queue(task_call{std::move(built.call), std::move(built.callback)});
00240 }
00241
00245 std::size_t pending() const {
00246     std::lock_guard<std::mutex> lock(mutex_);
00247     return pending_.size();
00248 }

```

```

00249
00257 void start() {
00258     // The workers first, so anything accepted after this is handed to one that is running.
00259     for (auto& one : workers_) {
00260         one->start();
00261     }
00262
00263     running_ = true;
00264 }
00265
00276 void stop() {
00277     // First, so that nothing is accepted for workers that are about to stop. A task already on a
00278     // worker still runs; one arriving from here is refused.
00279     running_ = false;
00280
00281     // No lock: a worker waking from stop() runs on_finished, which takes mutex_.
00282     for (auto& one : workers_) {
00283         one->stop();
00284     }
00285 }
00286
00308 std::function<void(std::exception_ptr)> on_task_error;
00309
00310 private:
00317 void report_task_error(const std::string& worker, std::exception_ptr thrown) {
00318     if (on_task_error) {
00319         on_task_error(thrown);
00320         return;
00321     }
00322
00323     try {
00324         std::rethrow_exception(thrown);
00325     } catch (const std::exception& e) {
00326         std::println(stderr, "warning: executor task on '{}' threw: {}", worker, e.what());
00327     } catch (...) {
00328         std::println(stderr, "warning: executor task on '{}' threw an unknown type", worker);
00329     }
00330 }
00331
00340 struct task_call {
00341     using result_type = typename actionT::result_type;
00342
00343     using callback_t = typename untangle::task_callback_type<result_type>::type;
00344
00345     result_type operator()() { return call(); }
00346
00347     std::function<result_type(void)> call;
00348
00349     callback_t callback;
00350 };
00351
00369 template <typename... Args>
00370 static task_call bind_action(actionT task, Args&&... args) {
00371     if constexpr (sizeof...(Args) == 0) {
00372         return task_call{std::move(task)};
00373     } else {
00374         // Captured by value and called as lvalues, which is all a call deferred to a worker can
00375         // promise: whatever was passed to add_action() may be gone by the time this runs.
00376         return task_call{{task = std::move(task), ... args = std::forward<Args>(args)}]() mutable ->
00377             typename task_call::result_type { return task(args...); }};
00378     }
00379 }
00380
00381 using worker_execution = untangle::async::execution<task_call>;
00382
00391 bool queue(task_call call) {
00392     std::lock_guard<std::mutex> lock(mutex_);
00393
00394     // The queue comes first: an entry does not overtake one already waiting in it.
00395     if (pending_.empty()) {
00396         // Always from the start, so a free pool reuses its warmest worker rather than waking a cold
00397         // one. Under load the workers are busy and the queue below is what spreads the work.
00398         for (std::size_t index = 0; index < workers_.size(); ++index) {
00399             if (!workers_[index]->is_busy()) {
00400                 // A refused entry is already destroyed, and stop() stops every worker together, so there
00401                 // is nothing to queue and no other worker to try.
00402                 return give_to_worker(index, std::move(call));
00403             }
00404         }
00405     }
00406
00407     pending_.push_back(std::move(call));
00408     return true;
00409 }
00410
00417 [[nodiscard]] bool give_to_worker(std::size_t index, task_call task) {
00418     // Both doors take the worker's action_mutex under mutex_. The two never cycle, because a

```

```

00419 // worker calls on_finished with its action_mutex released.
00420 if (task.callback) {
00421     // Taken out first: what the worker is handed is the callable, and the callback beside it as
00422     // the last argument, which is where async::execution::add_task() expects to find one.
00423     auto callback = std::move(task.callback);
00424     return workers_[index]->add_task(std::move(task), std::move(callback));
00425 }
00426
00427 return workers_[index]->add_action(std::move(task));
00428 }
00429
00433 void take_next_task(std::size_t index) {
00434     std::lock_guard<std::mutex> lock(mutex_);
00435
00436     if (!pending_.empty()) {
00437         task_call next = std::move(pending_.front());
00438         pending_.pop_front();
00439
00440         if (!give_to_worker(index, std::move(next))) {
00441             // The task is already destroyed, so it is reported rather than put back.
00442             std::println(stderr, "executor: {} refused a queued task, which is lost",
00443                 workers_[index]->name);
00444         }
00445
00446         return;
00447     }
00448
00449     if (nothing_running()) {
00450         finished_cv_.notify_all();
00451     }
00452 }
00453
00459 bool nothing_running() const {
00460     for (const auto& one : workers_) {
00461         if (one->is_busy()) {
00462             return false;
00463         }
00464     }
00465
00466     return true;
00467 }
00468
00474 std::size_t count_workers(bool (worker_execution::*state)() const) const {
00475     std::size_t count = 0;
00476
00477     for (const auto& one : workers_) {
00478         if ((one.get()->*state)()) {
00479             ++count;
00480         }
00481     }
00482
00483     return count;
00484 }
00485
00492 std::string name_workers(bool (worker_execution::*state)() const) const {
00493     std::string names;
00494
00495     for (const auto& one : workers_) {
00496         if ((one.get()->*state)()) {
00497             names += names.empty() ? " : " : ", ";
00498             names += one->name;
00499         }
00500     }
00501
00502     return names;
00503 }
00504
00506 static constexpr int poll_interval_ms = 50;
00507
00509 static constexpr int report_first_ms = 1000;
00510 static constexpr int report_max_ms = 30000;
00511
00512 mutable std::mutex mutex_;
00513 std::condition_variable finished_cv_;
00514
00520 async::execution_poll poll_;
00521
00523 std::deque<task_call> pending_;
00524 std::vector<std::shared_ptr<worker_execution> > workers_;
00526 std::atomic_bool running_ = {false};
00527 };
00528
00529 } // namespace untangle

```

Index

- ~executor
 - untangle::executor< actionT >, [16](#)
- add_action
 - untangle::executor< actionT >, [17](#)
- add_task
 - untangle::executor< actionT >, [17](#)
- executor, [1](#)
 - untangle::executor< actionT >, [16](#)
- on_task_error
 - untangle::executor< actionT >, [19](#)
- start
 - untangle::executor< actionT >, [18](#)
- stop
 - untangle::executor< actionT >, [19](#)
- untangle, [13](#)
- untangle::executor< actionT >, [15](#)
 - ~executor, [16](#)
 - add_action, [17](#)
 - add_task, [17](#)
 - executor, [16](#)
 - on_task_error, [19](#)
 - start, [18](#)
 - stop, [19](#)